Acid Properties Implementation

This document describes how the project implements and documents the four ACID properties: **Atomicity, Consistency, Isolation,** and **Durability.**

Atomicity:

Requirement: Minimum 2 transaction scenarios where all operations succeed completely or fail, with proper rollback.

Transaction Scenario 1: Match Request and Create Exchange + Session

Business flow: A student accepts a skill request. The system must:

1. Create an Exchange (linking Offer, Request, Conversation).

2. Create a Session for that exchange.

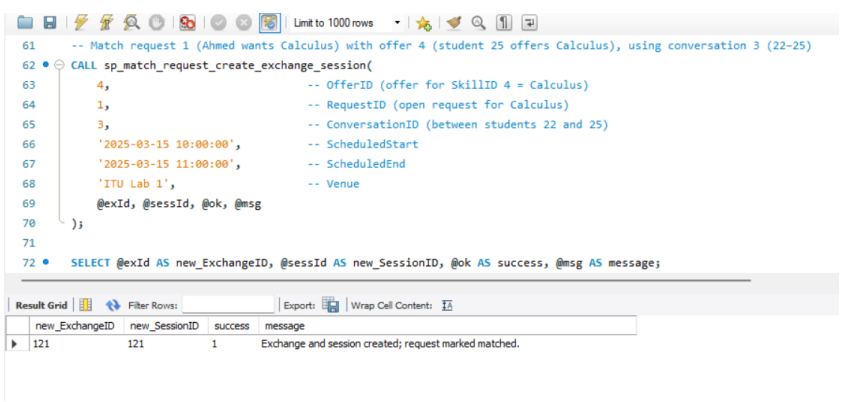
3. Update the RequestedSkill status to 'matched'.

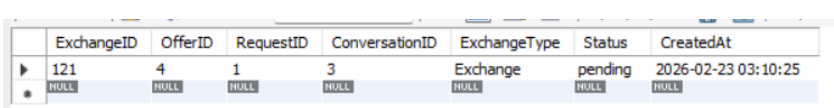
If any step fails (e.g., duplicate exchange, invalid FK), **nothing** should be committed: no new exchange, no session, and the request must stay 'open'.

Atomicity: Either all three operations are saved, or none are. Rollback on any error.

Step	Operation	Rollback if…
1	INSERT into Exchange	FK invalid, duplicate, constraint violation
2	INSERT into Session	ExchangeID invalid, time check fails
3	UPDATE RequestedSkill SET Status= 'matched'	Request already matched or not found

Implementation**:** See 'database/acid_transactions.sql' — procedure or transaction block for "Match request and create exchange + session".





ExchangeID	OfferID	RequestID	ConversationID	ExchangeType	Status	CreatedAt
121	4	1	3	Exchange	pending	2026-02-23 03:10:25
NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
75 •  SELECT * FROM Session  ORDER BY SessionID  DESC LIMIT 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content: | Fetch rows:

SessionID	ExchangeID	ScheduledStartTime	ScheduledEndTime	ActualStartTime	ActualEndTime	Venue	Status	CreatedAt
121	121	2025-03-15 10:00:00	2025-03-15 11:00:00	NULL	NULL	ITU Lab 1	scheduled	2026-02-23 03:10:25
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

```
77 •  SELECT RequestID, StudentID, SkillID, Status FROM RequestedSkill WHERE RequestID = 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

RequestID	StudentID	SkillID	Status
1	22	4	matched
NULL	NULL	NULL	NULL

First run: We got @ok = 1, and @exId / @sessId will be the new Exchange and Session IDs. Request 1 will be matched.

Second run (same request): We got @ok = 0 and something like “Request not found or not open”, because request 1 is already matched.

```
72 •  SELECT @exId AS new_ExchangeID, @sessId AS new_SessionID, @ok AS success, @msg AS message;
73    -- New exchange and session
74 •  SELECT * FROM Exchange ORDER BY ExchangeID DESC LIMIT 1;
75 •  SELECT * FROM Session  ORDER BY SessionID  DESC LIMIT 1;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

new_ExchangeID	new_SessionID	success	message
NULL	NULL	0	Request not found or not open.

Transaction Scenario 2: Paid Exchange with Payment Record

Business flow: For a paid exchange, the system must:

1. Insert a row into ‘PaidExchange’ (ExchangeID, Price, Currency).

2. Insert a row into ‘Payment’ (ExchangeID, Amount, PaymentStatus = 'completed', PaidAt).

3. Optionally update ‘Exchange.Status’ to ‘active’.

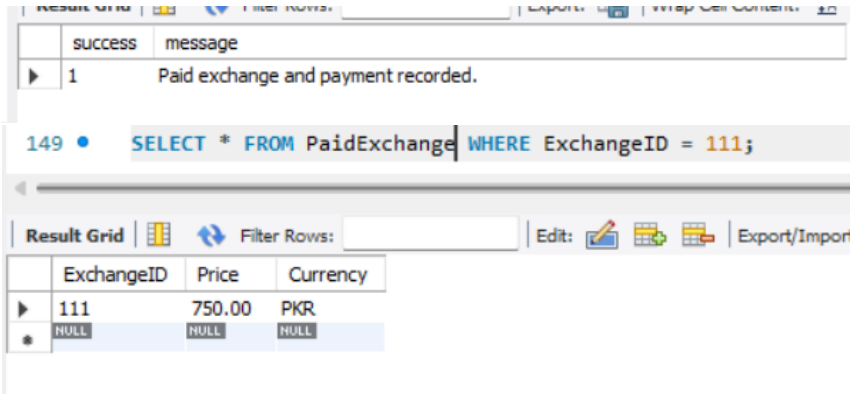
If payment insert fails or amount is invalid, the paid exchange record must not be committed either.

Atomicity: Either both PaidExchange and Payment are created together, or neither is created. Rollback on any error.

Step	Operation	Rollback if…
1	INSERT into PaidExchange	ExchangeID invalid, Price ≤ 0 (CHECK)
2	INSERT into Payment	ExchangeID invalid, Amount ≤ 0 (CHECK), duplicate payment logic
3	UPDATE Exchange (optional)	Exchange not found or invalid state

Implementation: See ‘database/acid_transactions.sql’ — procedure or transaction block for “Paid exchange with payment”.

The first time the transaction was run, it completed successfully:



But the second time it gave an error of duplicate entry which shows that there is atomicity:

CALL sp_create_paid_exchange_with_payment(111, -- ExchangeID (must exist; 111–120 are free and have no PaidExchange in seed) 750.00, -- Price (PKR) 'PKR', -- Currency 'JazCash', -- PaymentMethod @ok, @msg) Error Code: 1062. Duplicate entry '111' for key 'paidexchange.PRIMARY' 0.000 sec

Consistency:

Requirement: Implement database-level constraints (CHECK, FOREIGN KEY,UNIQUE) combined with triggers to maintain valid database states.

Constraints already in the schema:

Constraint Type	Example in Schema
Check	'chk_price' (OfferedSkill: PricePerHour IS NULL OR PricePerHour >= 0) 'chk_rating' (Review: Rating BETWEEN 1 AND 5), 'chk_payment_amount' (Payment: Amount > 0), 'chk_session_times' (Session: ScheduledEndTime > ScheduledStartTime)
Foreign key	All fk_* constraints (e.g. Exchange → OfferedSkill, RequestedSkill, Conversation)
Unique	'uq_user_email', 'uq_offer_slot' (OfferID, SlotStart), etc.

These ensure: positive prices/amounts, valid ratings, referential integrity, and no duplicate slots.

Triggers for consistency:

The following triggers are defined in 'database/schema.sql' and keep the database in a valid state:

Trigger	Table	When	What it does
trg_payment_validate	Payment	Before Insert	Ensures the exchange is a paid exchange ('Exchange.ExchangeType'= 'paid'). If not, raises **Cannot create payment for a non-paid exchange.** So payments cannot be linked to free exchanges.
trg_reputation_update	Review	After Insert	Adds the new review's 'Rating' to the reviewee's

			‘Student.ReputationPoints’. Keeps reputation in sync with reviews.
trg_slot_no_double_book	OfferTimeSlot	Before Update	If the slot was already ‘Status = ‘booked’’ and the update tries to set it to ‘booked’ again, raises **Time slot is already booked.** Prevents double-booking.
trg_exchange_cancel_sessions	Exchange	After Update	When an exchange’s ‘Status’ changes to ‘cancelled’, sets all its sessions (with status ‘scheduled’ or ‘ongoing’) to ‘cancelled’. Keeps sessions consistent with the exchange state.

Isolation:

Requirement: Handle concurrent transactions; document chosen isolation level and show understanding of dirty reads, non-repeatable reads, and lost updates.

Isolation means: when two users (or two transactions) use the database at the same time, one should not see half-finished or wrong data because of the other. We use an **isolation level** to control how strict this is.

Three Problems We Avoid:

1. **Dirty Read:** You read a row that another transaction changed but not yet committed. Later that transaction might roll back, so you saw data that never really happened.
   What we do: We do not use READ UNCOMMITTED. With REPEATABLE READ you do not get dirty reads.

2. **Non-repeatable read**: You read the same row twice in one transaction. Between the two reads, another transaction commits a change. So the second read gives a different value.
   What we do: With REPEATABLE READ the database gives you a consistent snapshot for the whole transaction, so you do not see other transactions' commits in the middle.

3. **Lost update**: Two transactions both read the same row, both change it, both write it back. The second write overwrites the first.
   What we do: We lock the row with ‘SELECT ... FOR UPDATE’ so only one transaction can update it until it commits or rolls back. We use this when matching a request so two users cannot both match the same request.

Where We Use Locking (FOR UPDATE)

When we match a request (create exchange + session + set request to matched), two users must not both match the same request. So we: (1) Start a transaction. (2) ‘SELECT ... FROM RequestedSkill WHERE RequestID = ? AND Status = 'open' FOR UPDATE’. (3) If a row is found, create exchange and session,

update request to matched, then COMMIT. (4) If not, ROLLBACK. **FOR UPDATE** locks that row until our transaction ends. Another session trying to lock the same row will wait; after we commit the request is already matched, so we avoid lost updates and double-matching.
Implementation: in ‘database/acid_transactions.sql’: how to set/check the isolation level (REPEATABLE READ), and the procedure ‘sp_match_request_with_lock’ that matches a request using ‘SELECT ... FOR UPDATE’.

```
224 •  CALL sp_match_request_with_lock(5, 3, 3, '2025-03-20 10:00:00', '2025-03-20 11:00:00', 'Lab 2', @exId, @sessId, @ok, @msg);
225 •  SELECT @exId, @sessId, @ok, @msg;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

@exId	@sessId	@ok	@msg
122	122	1	Exchange and session created; request marked matched (with row lock).

Durability:

Requirement: Committed transactions persist after system failure; use error handling and rollback so only committed work is durable.

Durability means: after you say commit, the database saves the changes for good. If the power goes off or the server restarts, your committed data is still there when the system comes back. Half-finished or failed work is not saved.

How the database does it: We use **InnoDB**. When you commit, InnoDB writes the changes to a **redo log** on disk. After a crash, InnoDB uses that log to restore the database to the last committed state. So only committed work survives; uncommitted or rolled-back work is lost, which is what we want.

What we have to do: We only call **COMMIT** when every step in the transaction succeeded. If anything goes wrong, we call **ROLLBACK** and never commit.

Error Handling and Rollback:

In stored procedures: We use an **EXIT HANDLER** for errors: when any SQL error happens, the handler runs **ROLLBACK** and then re-raises the error. So the procedure never commits after a failure. See the procedures in ‘acid_transactions.sql’ for the pattern.

In application code: Use try/catch. On success, call **COMMIT**. On catch (any error), call **ROLLBACK** and do not call COMMIT. Then log the error and return a safe message to the user.

That way only successful, complete transactions become durable; failed ones leave no partial data saved.